CS 7295 Project 2: CUDA Bitonic Sort

Kyle Nguyen
GTID: 903953383

Overview

For this project I implemented bitonic sort in CUDA and then optimized it for the H100 grading setup. Bitonic sort is not work-optimal compared with something like mergesort, but it is a good fit for GPUs because the compare-exchange pattern is regular. At each stage every element has a predictable partner, so I can run many independent comparators in parallel without data-dependent control flow.

My final version uses two main paths. Large-stride compare-exchanges still run through global-memory kernels, because those partners can cross tile boundaries. Once the stride becomes small enough to fit inside a tile, I switch to a shared-memory kernel and finish the remaining local merge steps inside one block. The final shipped configuration is:

Setting	Value
`DTYPE`	`uint16_t`
`BLOCK_DIM`	512
`TILE`	8192
Shared tile type	`int`

The main reason for `uint16_t` is simple: the input values are generated with `rand() % 1000`, so 16 bits are enough for every real value. This cuts H2D/D2H transfer size in half, while the shared-memory tile still uses `int` internally because that behaved better in profiling.

Implementation

The program first rounds the input size up to the next power of two, since bitonic sort assumes a power-of-two length. I allocate one device array and copy the real input values to it. The padded tail is filled on the device with `cudaMemset(..., 0xFF, ...)`, which produces `0xFFFF` for `uint16_t`. That value is safely above the largest real input value, so padding sorts to the end.

The baseline global idea is one compare-exchange per comparator pair. Instead of launching one thread per element and then throwing away the `k > partner` half, I map a pair index directly to the low element:

```
low = ((t >> j) << (j + 1)) | (t & ((1 << j) - 1))
partner = low + (1 << j)
```

This was a surprisingly important change. With the old element-indexed version, for the large global strides used in this code, entire blocks could launch and immediately return. Pair-indexing removes those dead blocks, which improved the Achieved Occupancy metric and also trimmed kernel time.

For small strides, `bitonic_shared` loads an 8192-element tile into shared memory, runs all remaining `j ... 0` compare-exchange steps locally, and writes the tile back once. This reduces both kernel launches and global-memory traffic. The global direction bit still uses the global index (`base + low`), so each tile participates in the same bitonic network as the full array.

I also made compare-exchange branchless:

```
lo = min(a, b)
hi = max(a, b)
arr[low]     = asc ? lo : hi
arr[partner] = asc ? hi : lo
```

This was not mainly a speed optimization. The timing did not move much by itself. The useful effect was that every comparator writes both outputs, which keeps the memory pipeline busier. That is what moved the Memory Throughput counter up in the `int` version.

Finally, after switching to `uint16_t`, the global compare kernel became the throughput bottleneck because each comparator was only moving 16-bit values. I added a packed global path for `j >= 1`: one thread handles two adjacent comparator pairs using aligned `uint32_t` loads/stores. The scalar path is kept for `j == 0`, where adjacent pairs cannot be packed safely the same way.

Optimization Log Summary

I kept a diary of the optimization attempts in `optimization.md`. The useful path was not one magic change, but a sequence of small corrections:

Change	Result
H2D `cudaHostRegister`	H2D dropped from about 42 ms to about 20 ms in the `int` version
D2H `malloc` + `cudaHostRegister`	D2H dropped from about 92 ms to about 64 ms in the `int` version
Move tail fill to kernel phase	Stopped charging padding work to H2D
`BLOCK_DIM = 512`	Slightly faster than 256 and better occupancy than 1024
Pair-index global kernel	Removed all-return blocks, improved occupancy and kernel time
Branchless compare-exchange	Raised memory-throughput counters by keeping stores active
`cudaMemset` tail padding	Removed the low-throughput custom fill kernel from the NCU kernel average
`TILE = 8192`	Fused more shared-memory steps and reduced kernel time
`DTYPE = uint16_t`	Main MEPS jump: transfer roughly halved and kernel time also improved
Packed `uint16_t` global kernel	Follow-up attempt for the global-kernel throughput bottleneck

Some attempts were rejected. `int4` vectorized shared-memory loads/stores looked promising, but it created shared-memory bank conflicts and made the kernel slower. Register-blocking inside `bitonic_shared` also regressed because the kernel already had high enough occupancy; extra per-thread ILP just increased register pressure. I also analyzed skipping padding-only comparators, but position-based skipping is not correct because real values can temporarily move into the padded region during descending phases.

Performance Results

All correctness tests passed for the graded sizes: 2K, 10K, 100K, 1M, and 10M. The most important final performance result is the 100M-element throughput:

Metric	Final measured value
H2D transfer	8.216 ms
Kernel time	44.629 ms
D2H transfer	32.226 ms
Total transfer	40.442 ms
MEPS	1175.491
Performance option	Option 1, full credit

The `uint16_t` change is what flipped the project from an Option-2 solution into an Option-1 solution. Before that, the `int` version was fighting a transfer floor: even after pinning, it was moving about 400 MB each direction at 100M elements. With 16-bit elements, the transfer volume is about half, and the MEPS score crosses the 1000 MEPS full-credit target.

The profiler counters tell a slightly different story. Before `uint16_t`, branchless compare-exchange plus `cudaMemset` got Memory Throughput just over the threshold, and pair-indexing got Achieved Occupancy over the threshold. In the `dtype_int16.log` run, Achieved Occupancy still passed at **82.41%**, but Memory Throughput dropped to **55.65%** because the global compare kernel moved fewer bytes per comparator.

Counter	Status
Achieved Occupancy	82.41%, passed

Counter	Status
Memory Throughput after `uint16_t`	55.65%
Memory Throughput target	75%

My read is that the remaining miss is very localized. `bitonic_shared` is no longer the problem; it already benefits from shared-memory reuse and the branchless writes. The global compare kernel is the drag because the final data type is 16-bit. That is why my follow-up work focused on packing adjacent `uint16_t` comparator pairs into wider loads/stores, but the logged result above is still the clean reference point for the `uint16_t` change.

Conclusion

The final implementation gets the main performance target through a combination of real algorithm work and transfer reduction. Shared-memory fusion reduced the number of global passes. Pair-indexing removed wasted global-kernel blocks and helped occupancy. Branchless compare-exchange improved the profiler's memory throughput behavior. Pinned host memory reduced copy time. The largest single MEPS gain came from changing the stored element type to `uint16_t`, which is valid for this input generator and cuts the transfer volume in half.

The current result is strong but not perfect: correctness passes, MEPS is full credit, and Achieved Occupancy passes. Memory Throughput is still below the 75% extra-credit cutoff after the `uint16_t` switch. If I had more time, I would keep working specifically on the global `uint16_t` compare path, either by packing wider where alignment allows it or by testing an internal 32-bit work buffer while keeping 16-bit H2D/D2H transfers.